PBA: 5. Causality II: DAGs and Covariate Selection

Fall 2026

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Where are we? Where are we going?

- **Where we have been**
  - Weeks 1–3: R and RStudio, data visualization with `ggplot2`, data wrangling with `dplyr`
  - Week 4: causality. Potential outcomes, randomized experiments, and three designs for observational data (Line Pay)

Where are we? Where are we going?

- **Where we have been**
  - Weeks 1–3: R and RStudio, data visualization with `ggplot2`, data wrangling with `dplyr`
  - Week 4: causality. Potential outcomes, randomized experiments, and three designs for observational data (Line Pay)
- **Today: causality II**
  - Causal diagrams (DAGs): which variables to control for, and which not to
  - Building on Week 4: the confounding picture and `group_by()` comparisons, with data we simulate ourselves, so we know the true effect and can see which comparison gets it right
  - After the lecture you run today's code yourself, then the DAG practice at your own pace

Where are we? Where are we going?

- **Where we have been**
  - Weeks 1–3: R and RStudio, data visualization with `ggplot2`, data wrangling with `dplyr`
  - Week 4: causality. Potential outcomes, randomized experiments, and three designs for observational data (Line Pay)
- **Today: causality II**
  - Causal diagrams (DAGs): which variables to control for, and which not to
  - Building on Week 4: the confounding picture and `group_by()` comparisons, with data we simulate ourselves, so we know the true effect and can see which comparison gets it right
  - After the lecture you run today's code yourself, then the DAG practice at your own pace
- **Where we are going**
  - Team meetings by appointment, ideally by Wednesday, Oct 21: bring your research question, its context, and your DAG
  - Week 7 (Oct 22): relationships between variables, importing and tidying data. Then prediction, regression, and uncertainty

Roadmap

1. DAGs: pictures of what causes what

2. Reading a DAG: forks, chains, colliders

3. Choosing controls for your project

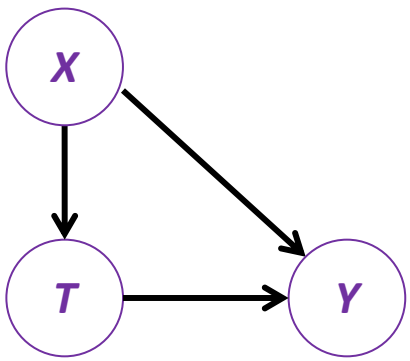
4. In-Class Practice: Drawing DAGs



Sources: https://www.redbubble.com/i/sticker/D-Ya-Like-Dags-by-salamincheese/27407958.EJUG5

p.s.: from an old British-American comedy film starring Brad Pitt

Week 4: Confounding in the Line Pay Data



- X: tech-savviness, T: Line Pay adoption, Y: m-commerce spending
- Adopters vs. non-adopters, after the launch: **+23.4**
- Within tech-savviness groups (subclassification): **+20.1** and **+19.0**

- This picture is already a **causal diagram**: arrows say what causes what.
- Comparing within groups of X removed the part of the gap that came through X.
- Today: the grammar of such pictures, so that you can decide which variables to compare within, and which ones to leave alone.

Today's Questions

- Which variables should we **control for**?
- Which variables should we not control for, even though they are in the data?
- What is a **pre-treatment** variable, and is "pre-treatment" enough?
- A dataset with dozens of columns: **how do we choose**?

1/ DAGs: pictures of what causes what

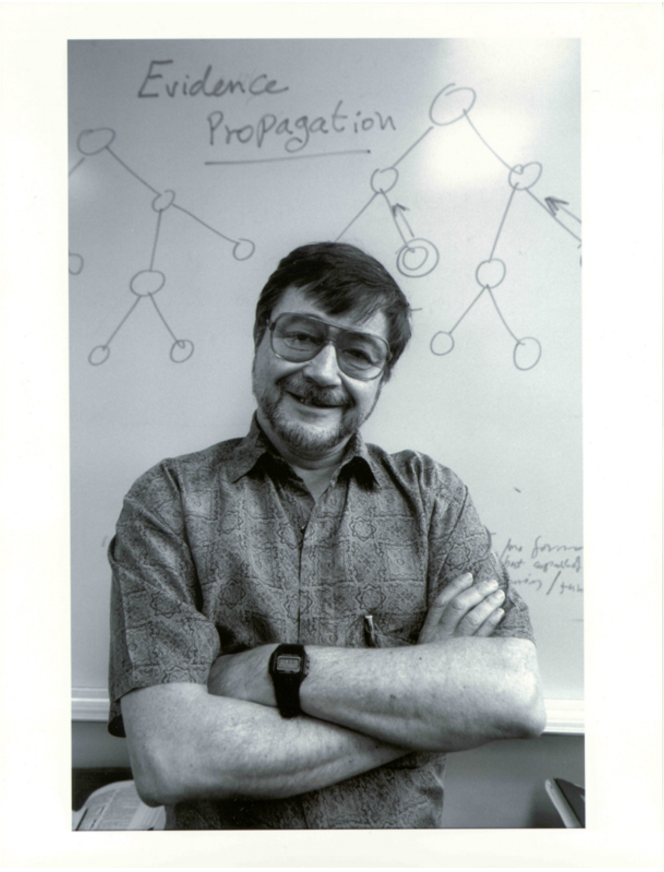
Recap: Potential Outcomes

In Week 4 we defined causal effects as contrasts between potential outcomes:

$$Y_i(1) - Y_i(0), \qquad \text{SATE} = \frac{1}{n}\sum_{i=1}^{n}\big(Y_i(1) - Y_i(0)\big)$$

- We never see both $Y_i(1)$ and $Y_i(0)$ for the same unit (fundamental problem).
- Randomization makes treated and control groups comparable *by design*.
- Without randomization, we need assumptions: e.g., *comparable within groups of tech-savviness*.
- Today: an **alternative causal language**, directed acyclic graphs (DAGs).
  - Same goal (learn $Y_i(1) - Y_i(0)$ on average). In Week 4 we stated assumptions in words; a DAG draws them as arrows.

Judea Pearl and Causal Graphs



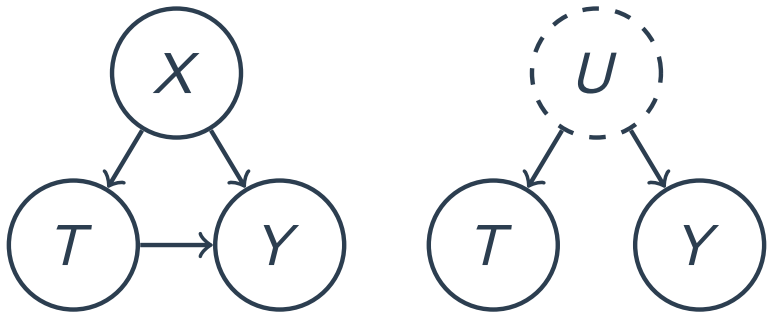
Judea Pearl (1936–) turned causal diagrams into a formal language of causality.

- Originally a computer scientist (artificial intelligence)
- *Causality* (2000, Cambridge UP)
- Turing Award in 2011 for this work

Key idea: no functional form. A graph only says *which* variables each variable depends on, not *how*.

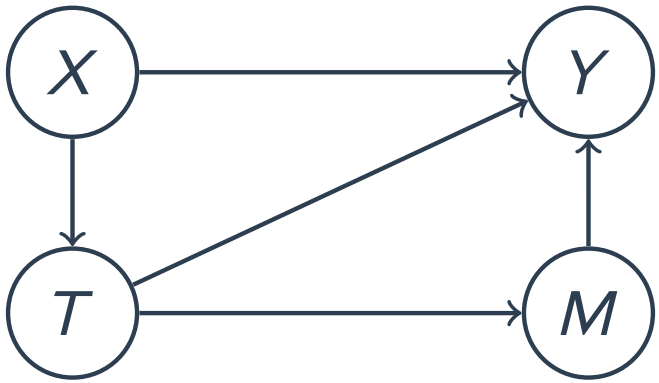
Directed Acyclic Graphs

- **Directed acyclic graphs** (DAGs) describe the causal structure of variables



- **Nodes**: variables, observed (solid) or unobserved (dashed).
- **Edges** (arrows): a direct causal effect.
  - Arrow $T \longrightarrow Y$ (left): $Y_i(1) \neq Y_i(0)$ for at least some units.
  - No arrow $T \longrightarrow Y$ (right): $Y_i(1) = Y_i(0)$ for every unit.
  - A missing arrow is an assumption: no direct effect at all.
- **Missing variables**: any common cause of variables in the graph must be drawn too (dashed if unobserved).
- **Directed**: each arrow has a direction (cause $\longrightarrow$ effect).
- **Acyclic**: no cycles. A variable cannot cause itself.

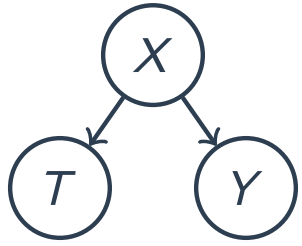
DAG Terminology



- **Path**: a sequence of arrows connecting two nodes, in any direction.
  - **Causal (directed) path**: all arrows point the same way, e.g., $T \to M \to Y$.
  - **Non-causal path**, e.g., $T \leftarrow X \to Y$.
- **Parents** and **children**: T is a parent of M and Y; M is a child of T.
- **Ancestors** and **descendants**: M and Y are descendants of T and X; X is an ancestor of M.

A DAG Is a Recipe for Data

```
> set.seed(1008)
> n <- 2000
> tech  <- rbinom(n, 1, 0.5)                # X: no parents
> adopt <- rbinom(n, 1, 0.3 + 0.4 * tech)   # T: parent X
> spend <- 50 + 20 * tech + rnorm(n, 0, 10)  # Y: parent X only
> fork  <- tibble(tech, adopt, spend)
```



- **One line per node**: each variable is made from its parents plus its own noise.
  - `rbinom(n, 1, p)`: one coin flip (size = 1) for each of n customers, with probability p of landing on 1 (e.g., "tech-savvy"). `rnorm(n, 0, 10)`: n random numbers with mean 0 and standard deviation 10.
- **Arrows in the DAG**: each variable gets an arrow from every variable used in its line. The line creating spend does not use adopt, so there is no arrow $T \to Y$.
  - In potential outcomes: $Y_i(1) = Y_i(0)$ for every customer. In this made-up example, adopting a retailer's app has **zero effect**, and we know it.
- Real data come from a recipe we never see. Here we write the recipe, so we know the true effect and can see which comparison (overall or within groups) gets it right.

Q: Draw the DAG

Each recipe below has three lines. Each line creates one variable. In the DAG, draw an arrow into each variable from every variable used in its line.

Recipe A

```
adopt  <- rbinom(n, 1, 0.5)  # a retailer's app
coupon <- rbinom(n, 1, 0.2 + 0.5 * adopt)
spend  <- 50 + 15 * coupon + rnorm(n, 0, 10)
```

Recipe B

```
adopt  <- rbinom(n, 1, 0.5)  # a retailer's app
spend  <- 50 + rnorm(n, 0, 10)
member <- if_else(adopt == 1 | spend > 60,
                  1, 0)
```

Q: Draw the DAG

Each recipe below has three lines. Each line creates one variable. In the DAG, draw an arrow into each variable from every variable used in its line.

Recipe A

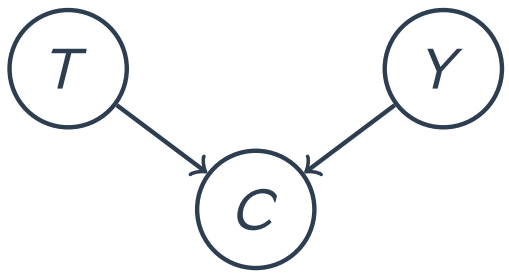
```
adopt  <- rbinom(n, 1, 0.5)  # a retailer's app
coupon <- rbinom(n, 1, 0.2 + 0.5 * adopt)
spend  <- 50 + 15 * coupon + rnorm(n, 0, 10)
```



M: uses in-app coupons

Recipe B

```
adopt  <- rbinom(n, 1, 0.5)  # a retailer's app
spend  <- 50 + rnorm(n, 0, 10)
member <- if_else(adopt == 1 | spend > 60,
                  1, 0)
```

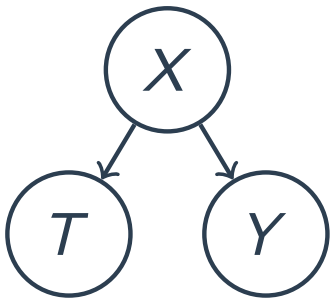


C: joins the retailer's loyalty program (a customer joins if they adopt the app *or* spend over 60)

2/ Reading a DAG: forks, chains, colliders

Three Building Blocks

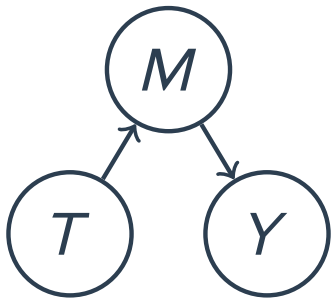
Fork



X = **confounder**

common cause

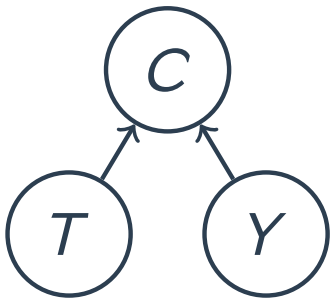
Chain



M = **mediator**

on the causal path

Inverted fork



C = **collider**

common effect

- Every larger DAG is built from these three pieces.
- For each one, two questions:
  1. Q1. Do T and Y move together in the data?
  2. Q2. Do they move together once we **compare within groups** of the middle variable?

What Does "Independent" Mean?

Q1′. Is $T \perp\!\!\!\perp Y$? **Independent**: knowing T tells us nothing about Y. Not independent = they move together.

- Quick check with the tools from Week 4: does the average of Y differ between $T = 1$ and $T = 0$?
- An average is one summary of Y's distribution: a gap shows dependence, but equal averages alone do not prove independence.

Q2′. Is $T \perp\!\!\!\perp Y \mid X$? **Conditionally independent**: independent *within each group* of X.

- "Conditioning on X" = "controlling for X" = comparing within groups of X (Week 4: subclassification).

Fork in Data: Overall Comparison

- Recipe from before: tech-savvy customers adopt more *and* spend more; the app itself does nothing.

```
> fork |>
    group_by(adopt) |>
    summarize(avg_spend = mean(spend))

# A tibble: 2 × 2
  adopt avg_spend
  <int>     <dbl>
1     0      55.8
2     1      64.2
```

- Adopters spend **8.4 more**, although adoption has no effect in this recipe.
- Q: Where does the gap come from?

Fork in Data: Compare Within Tech Groups

```
> fork |>
    group_by(tech, adopt) |>
    summarize(avg_spend = mean(spend)) |>
    pivot_wider(names_from = adopt, values_from = avg_spend,
                names_prefix = "adopt_") |>
    mutate(diff = adopt_1 - adopt_0)

# A tibble: 2 × 4
# Groups:   tech [2]
   tech adopt_0 adopt_1    diff
  <int>   <dbl>   <dbl>   <dbl>
1     0    50.3    50.1 -0.230
2     1    69.8    69.9  0.0786
```

- Within each tech group the gap is **close to zero**. This fits the recipe, where $T \perp\!\!\!\perp Y \mid X$.
- Fork: controlling for the confounder removes a gap that is not causal. This is what subclassification did for Line Pay in Week 4.

Chain in Data: Overall Comparison

```
> set.seed(1008)
> adopt  <- rbinom(n, 1, 0.5)                    # T: no parents
> coupon <- rbinom(n, 1, 0.2 + 0.5 * adopt)      # M: parent T
> spend  <- 50 + 15 * coupon + rnorm(n, 0, 10)   # Y: parent M only
> chain  <- tibble(adopt, coupon, spend)

> chain |>
    group_by(adopt) |>
    summarize(avg_spend = mean(spend))

# A tibble: 2 × 2
  adopt avg_spend
  <int>     <dbl>
1     0      53.0
2     1      60.4
```

- Here the app **does** raise spending, through coupon use: adopters use more coupons, and coupon users spend more.
- The gap of 7.4 is a real causal effect (the recipe implies $0.5 \times 15 = 7.5$).

Chain in Data: Compare Within Coupon Groups

```
> chain |>
    group_by(coupon, adopt) |>
    summarize(avg_spend = mean(spend)) |>
    pivot_wider(names_from = adopt, values_from = avg_spend,
                names_prefix = "adopt_") |>
    mutate(diff = adopt_1 - adopt_0)

# A tibble: 2 × 4
# Groups:   coupon [2]
  coupon adopt_0 adopt_1   diff
   <int>   <dbl>   <dbl>  <dbl>
1      0    50.3    49.8 -0.522
2      1    65.0    64.9 -0.135
```

- Within coupon groups the gap **disappears**, yet the effect is real: in this recipe all of it runs through coupons.
- Chain: controlling for the mediator removes the part of the effect that runs through it.
  - Want the **total** effect? Do not control for M. Want the **direct** effect (not through M)? Control for M.

Collider in Data: Overall Comparison

```
> set.seed(1008)
> adopt  <- rbinom(n, 1, 0.5)                        # T: no parents
> spend  <- 50 + rnorm(n, 0, 10)                     # Y: no parents
> member <- if_else(adopt == 1 | spend > 60, 1, 0)   # C: parents T and Y
> collider <- tibble(adopt, spend, member)

> collider |>
    group_by(adopt) |>
    summarize(avg_spend = mean(spend))

# A tibble: 2 × 2
  adopt avg_spend
  <int>     <dbl>
1     0      49.7
2     1      50.1
```

- No effect in the recipe, and **almost no gap** in the data. In this recipe, $T \perp\!\!\!\perp Y$.
- Now suppose the only data we can get come from the retailer's **loyalty program**.

Collider in Data: Analyze Members Only

```
> collider |>
    filter(member == 1) |>
    group_by(adopt) |>
    summarize(avg_spend = mean(spend), n = n())

# A tibble: 2 × 3
  adopt avg_spend     n
  <int>     <dbl> <int>
1     0      65.2   144
2     1      50.1   985
```

- Among members, adopters spend **15.1 less**. But in the recipe, adopting has no effect on spending.
- Why: a non-adopter is a member *only if* they spend over 60. Selecting on the collider links its two causes.
- `filter()` on a collider is conditioning too.

Collider in Data: The Same Customers, Two Views



- Black diamonds: group averages. Keeping only members removes the low-spending non-adopters.

Three Structures, Three Patterns

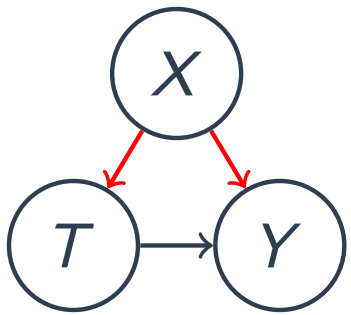
Structure	Overall gap	Gap within groups
Fork $T \leftarrow X \rightarrow Y$	yes (not causal)	gone
Chain $T \rightarrow M \rightarrow Y$	yes (causal)	gone
Collider $T \rightarrow C \leftarrow Y$	none	appears

- A path through a fork or a chain is **open** until we condition on the middle node.
- A path through a collider is **blocked** until we condition on the collider.
- Whether a variable is a collider depends on the path: the same variable can be the middle of a chain on one path and a collider on another.
- These two rules, applied to every path, are called d-separation (Appendix).

3/ Choosing controls for your project

Backdoor Paths

- The causal effect of T on Y travels along the **causal paths** out of T.
- A **backdoor path** is a non-causal path from T to Y that starts with an arrow *into* T.
  - It would remain even if we deleted every arrow coming out of T.
  - Open backdoor paths come from **common causes** of T and Y.



- Line Pay: $T \leftarrow X \rightarrow Y$ (adoption $\leftarrow$ tech-savviness $\rightarrow$ spending).
- The overall gap mixes the causal path $T \rightarrow Y$ with the open backdoor path.

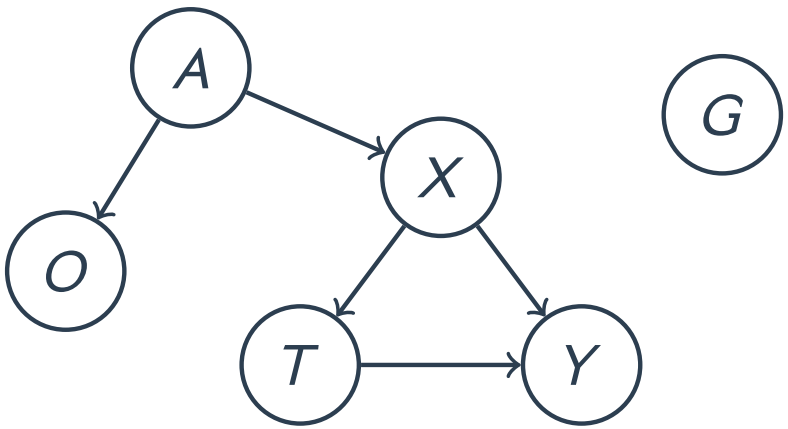
The Backdoor Criterion

A set of variables S is enough to control for (to get the total effect of T on Y) if

1. no variable in S is a **descendant of** T (no post-treatment variables), and
2. S **blocks every backdoor path** from T to Y.

- Then treated and control units are comparable *within each group formed by* S (for example, tech-savvy and not tech-savvy customers). This is the Week 4 assumption, and the DAG now tells us why it holds.
- The criterion answers three questions at once:
    1. Is there confounding, given this DAG?
    2. Can it be removed with the variables we have?
    3. Which variables should we control for?

Line Pay, Revisited

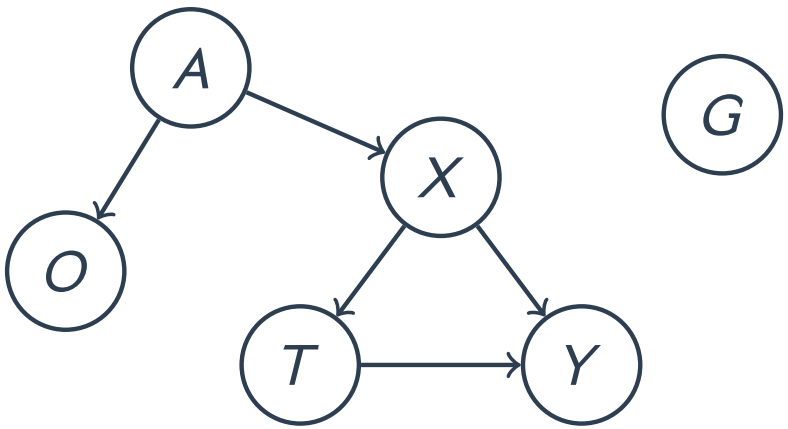


Our assumed DAG for the `linepay` data.

X: tech-savviness, T: adoption, Y: spending after the launch, A: age, O: occupation, G: gender.

- Q: Which variables satisfy the backdoor criterion?

Line Pay, Revisited



Our assumed DAG for the `linepay` data.

X: tech-savviness, T: adoption, Y: spending after the launch, A: age, O: occupation, G: gender.

- Q: Which variables satisfy the backdoor criterion?
- `tech_savviness`: **yes**, it blocks $T \leftarrow X \rightarrow Y$.
- `gender`: **no**, not on any backdoor path.
- age: **no** on its own. It causes X but does not sit on $T \leftarrow X \rightarrow Y$. Once X is in, age adds nothing.
- `occupation_type`: **no**, not on any backdoor path.

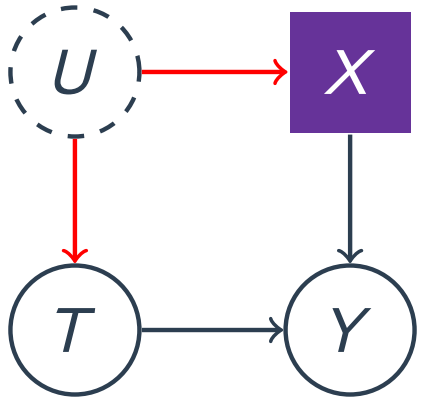
Line Pay, Revisited: Check With the Data

```
> LinePay_data |>
    group_by(gender, line_pay_adopt) |>
    summarize(avg_spending = mean(spending_post)) |>
    mutate(line_pay_adopt = if_else(line_pay_adopt == 1, "adopted", "unadopted")) |>
    pivot_wider(names_from = line_pay_adopt, values_from = avg_spending) |>
    mutate(diff = adopted - unadopted)

# A tibble: 2 × 4
# Groups:   gender [2]
  gender unadopted adopted  diff
   <int>     <dbl>   <dbl> <dbl>
1      0      59.6    82.3  22.7
2      1      58.9    82.9  24.0
```

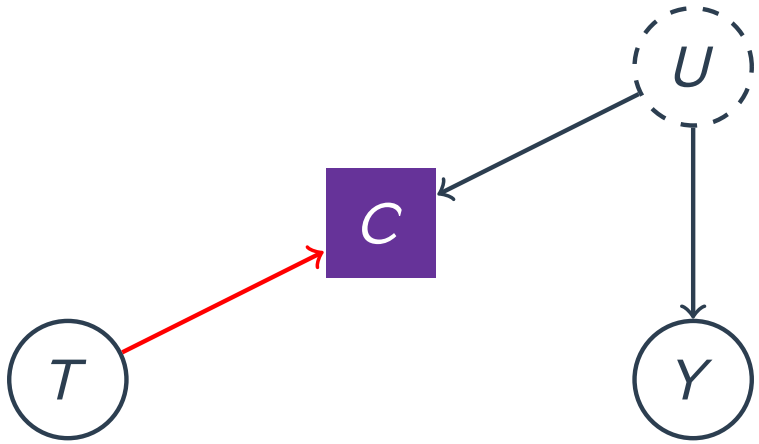
- Within gender groups: **22.7** and **24.0**, still close to the overall 23.4.
- Within tech-savviness groups (Week 4): **20.1** and **19.0**.
- The `linepay` data were simulated with an effect of **+20** for every adopter. Only tech-savviness, the variable the DAG told us to control for, gives estimates close to +20.

Can't Measure the Confounder? Block the Path



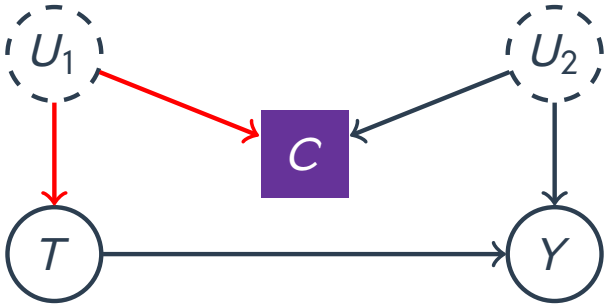
- T: job training program. Y: landing a job. U: motivation (unobserved). X: number of applications sent.
- Backdoor path $T \leftarrow U \rightarrow X \rightarrow Y$. We cannot measure U, but conditioning on X (shaded) **blocks** the path.
- Big assumption: no arrow $U \rightarrow Y$. Otherwise no observed variable blocks $T \leftarrow U \rightarrow Y$.

Bad Control: Post-Treatment Variables



- T: adopts the retailer's app. C: app log-ins in the month *after* adoption. U: shopping enthusiasm (unobserved). No arrow $T \longrightarrow Y$.
- C is a collider on $T \longrightarrow C \longleftarrow U \longrightarrow Y$. Controlling for C opens the path: a gap appears where there is no effect.
- A **pre-treatment variable** is one the treatment cannot affect: in the DAG, it is not a descendant of T.

Is Pre-Treatment Always Safe? M-Bias



- Without conditioning, $T \leftarrow U_1 \rightarrow C \leftarrow U_2 \rightarrow Y$ is **blocked** by the collider C.
- Conditioning on C opens it, even though C is a pre-treatment variable (not a descendant of T). (The graph's shape gives the name.)
  - *Ex*: T = schooling, Y = earnings, U_1 = family background, U_2 = ability, C = a test score. This example assumes that ability does not affect schooling.
- Controversial: should we always control for pre-treatment variables?
  - Rubin: yes. Exact M shapes are rare, like "trying to balance a multidimensional cone on its point."
  - Pearl: not always. M-bias is a real threat; what decides is the variable's place in the DAG, not only when it was measured ("structure, not timing").

Good Controls, Bad Controls

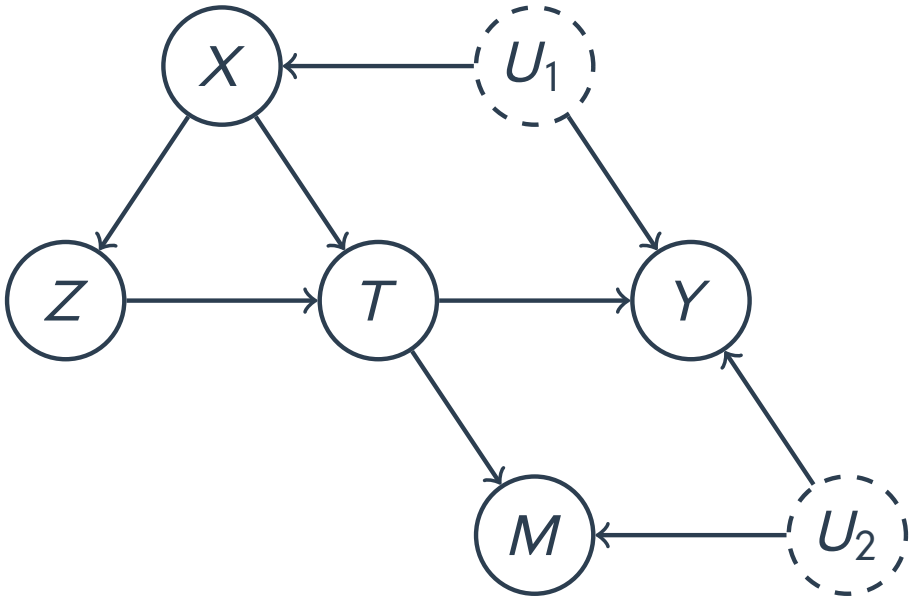
"Should I control for this variable?" has no one-size-fits-all answer.

Cinelli, Forney, & Pearl (2022) sort the answers by the variable's role in the DAG (target: the total effect of T on Y):

Type	Role in the DAG	Effect of controlling for it
Good	Confounder (common cause of T, Y)	Removes confounding bias
Neutral	Cause of Y only	No bias; may improve precision
Neutral	Cause of T only	No bias; may reduce precision
Bad	Collider (common *effect* of T, Y)	Opens a non-causal path
Bad	Mediator ($T \longrightarrow M \longrightarrow Y$)	Removes part of the effect
Bad	Descendant of Y	Post-treatment bias

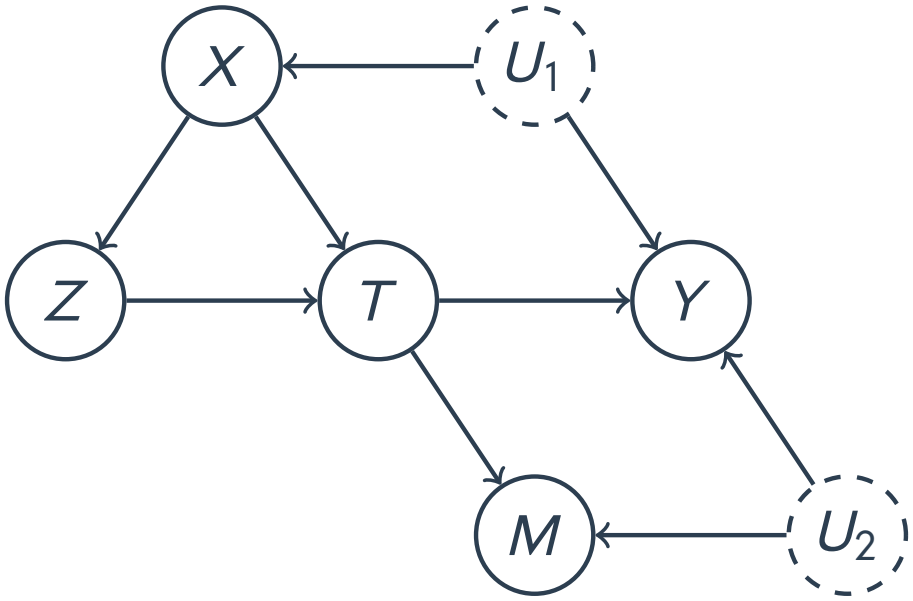
- Good or bad depends on the variable's **role in the DAG**, not on patterns in the data. So do not control for every variable you have.

Practice: The Backdoor Criterion by Hand



1. List all **backdoor paths** between T and Y.
2. List the sets of *observed* nodes that **block** all of them.
3. Drop the sets that include a **descendant of** T.

Practice: The Backdoor Criterion by Hand



1. List all **backdoor paths** between T and Y.
2. List the sets of *observed* nodes that **block** all of them.
3. Drop the sets that include a **descendant of** T.

- Backdoor paths: $T \leftarrow X \leftarrow U_1 \rightarrow Y$ and $T \leftarrow Z \leftarrow X \leftarrow U_1 \rightarrow Y$. Blocking sets: $\{X\}$, $\{X, Z\}$. Leave M out: it is a descendant of T, and a collider. Here M is not a mediator: it does not cause Y.

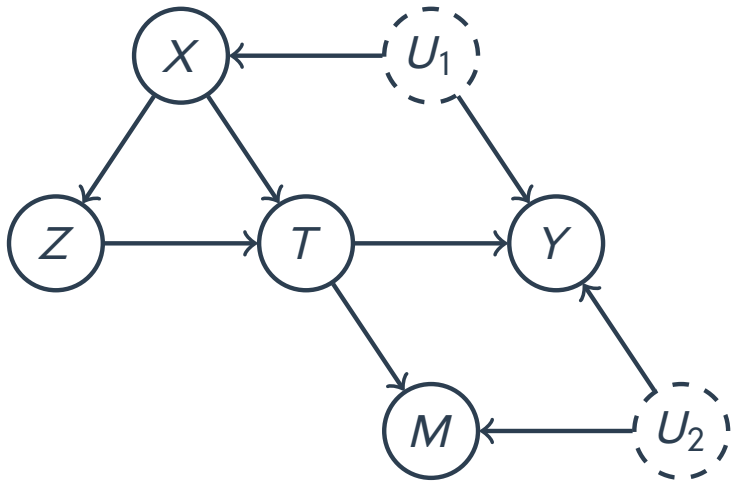
Why Let the Computer Do It?

Backdoor analysis by hand is fine for a small DAG like the one on slide 31. Project DAGs are bigger.

- **Scale**: with 15 nodes, the number of paths grows quickly.
- **Correctness**: people miss paths where colliders and chains interact.
- **Reproducibility**: a DAG written as code can be checked by your teammates and by me.
- **Robustness**: change one arrow, rerun, see whether the answer survives.

R Package: dagitty

```
# install.packages("dagitty")  # once
> library(dagitty)
> g <- dagitty('dag {
      X -> Z -> T -> Y
      X -> T -> M
      M <- U2 -> Y
      X <- U1 -> Y
  }')
> latents(g) <- c("U1", "U2")   # unobserved
```



- Each line lists arrows; `X -> Z -> T -> Y` is three arrows at once.
- `plot(g)` draws it; `https://www.dagitty.net` draws it in the browser.

R Package: dagitty

- Parents and descendants:

```
> parents(g, "T")
[1] "X" "Z"
> descendants(g, "T")
[1] "T" "Y" "M"
```

- All paths between T and Y, and only the causal ones:

```
> paths(g, "T", "Y")$paths
[1] "T -> M <- U2 -> Y"      "T -> Y"                "T <- X <- U1 -> Y"
[4] "T <- Z <- X <- U1 -> Y"
> paths(g, "T", "Y", directed = TRUE)$paths
[1] "T -> Y"
```

- d-separation (Appendix):

```
> dseparated(g, "Z", "M", c("T"))
[1] TRUE
```

R Package: dagitty

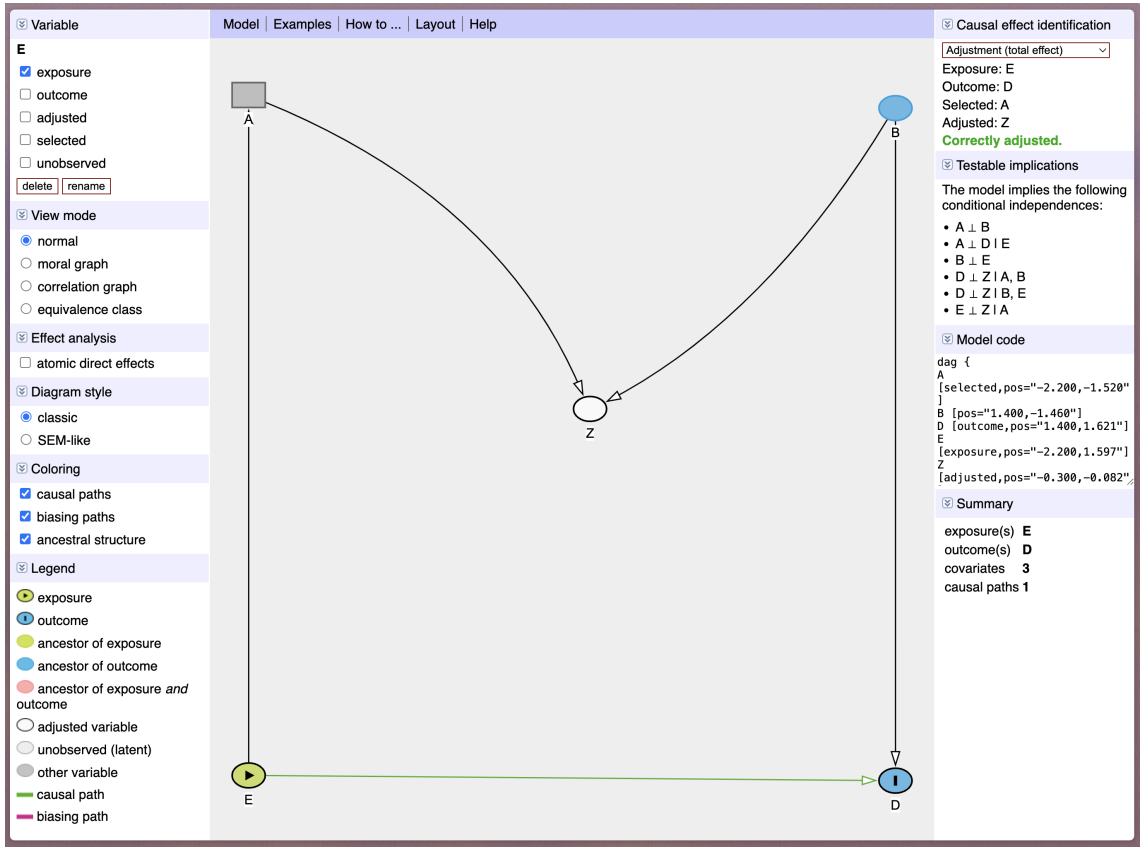
- What to control for (dagitty calls this an **adjustment set**):

```
> adjustmentSets(g, "T", "Y")
{ X }
```

- The same answer we found by hand. `type = "all"` lists all 18 sets that dagitty accepts.

- Which set to use?
  - Sets with U_1 or U_2 are not available: they are unobserved.
  - That leaves $\{X\}$ and $\{X, Z\}$.
  - The smallest observed set is the cheapest to collect and is enough to remove the confounding.
  - Z affects Y only through T, so adding Z may reduce precision (slide 30).

DAGitty in the Browser (dagitty.net)



- Draw interactively; valid adjustment sets are highlighted as you go; exports the code for R.

Where Does the DAG Come From?

- Not from the data. From **theory, domain knowledge, and how the data were produced**.
- A practical workflow:
  1. **Nodes**: T, Y, and every variable that plausibly causes either (unobserved $\rightsquigarrow$ dashed).
  2. **Arrows**: for each pair, "could A directly cause B?" Leaving an arrow out assumes no direct effect at all.
  3. **Direction**: temporal ordering, manipulability, natural laws.
  4. **How rows enter the data**: any selection step is a node too.
- Drawing the DAG "forces a clearer articulation of the research question" (Tafti & Shmueli 2025, *MIS Quarterly*).
- If you are not sure about an arrow, draw both versions and see whether your answer changes.

What a DAG Cannot Do

- A DAG is only as good as its assumptions. Data can reject some of them, but never confirm the whole graph.
- If an open backdoor path has **no observed variable** we can control for (like $T \leftarrow U \rightarrow Y$), no set of observed controls is enough.
  - Then we need designs that handle confounding differently: experiments, or designs such as difference-in-differences from Week 4.

Summing Up

- A DAG is an **alternative causal language** to potential outcomes: same goal, different way of stating assumptions.
- Three building blocks: **fork** (control for it), **chain** (do not for the total effect; do for the direct effect), **collider** (do not control or select on it).
- **d-separation** applies these rules to every path; the **backdoor criterion** turns them into a list of controls.
- "Should I control for X?" is a **structural** question, answered by the DAG, not by the data.
- `group_by()` and `filter()` are conditioning. So is choosing which rows enter your data.

4/ In-Class Practice: Drawing DAGs

In-Class Practice: Background

- With your team, draw a DAG for this question:
  - **Does using generative AI for assignments raise a student's final grade?**
- If your team already has its own causal question, you can draw a DAG for it instead.
- The next slide lists the variables. You decide the arrows.

In-Class Practice: Task

- Draw the DAG with these variables. (U) = unobserved.
  - T: uses generative AI for assignments (yes or no)
  - Y: final grade
  - GPA before the course
  - hours spent on assignments
  - asked the TA for help (during the course)
  - motivation (U)

- Then answer: which variables should you control for? Which must you **not** control for, and why?

In-Class Practice: Instructions

1. Open `https://www.dagitty.net/dags.html` and choose **Model** → **New Model**. Enter your T as the exposure and your Y as the outcome.
2. To add a variable, double-click on an empty spot. To mark it unobserved, point at it and press u.
3. To add an arrow, double-click the cause, then the effect. Draw an arrow only where you believe a direct effect exists.
4. Read the adjustment set in the box **Causal effect identification** (top right). Compare it with the table on slide 30.
5. Write down which variables you must **not** control for, and why.
6. Write one sentence on the unobserved variable that worries you most.

Q: If you are not sure about an arrow, draw both versions. Does your answer change?

What Comes Next?

- Today: causal diagrams, three building blocks, and the backdoor criterion.

- Next week (Oct 22): **relationships, importing, and tidying data**. Complete the Required reading in the weekly schedule beforehand.
  - Correlation measures how variables move together; today's DAG tells you what that movement can and cannot mean.

- Team meetings by appointment, ideally by Wednesday, Oct 21: bring your research question, its context, and your DAG.
  - No question yet? Start from the guide to finding data: `https://j1yoo.github.io/teaching/pba-data/`
  - Drawing your DAG: Huntington-Klein, *The Effect*, Ch. 7 (free online).

- Milestone 2 (data and proposal) is due Sunday, Oct 25.

Enjoy your Weekends! :)

Contact Information:
jaewon.yoo@iss.nthu.edu.tw
https://j1yoo.github.io/



Appendix: *d*-Separation

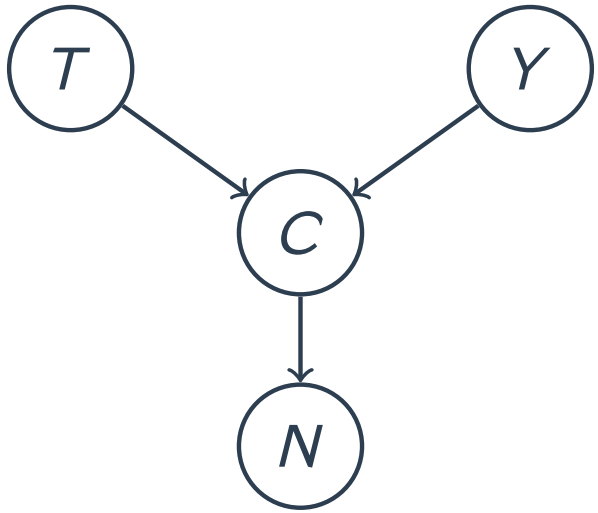
d-Separation

- Can we read (conditional) independence off a DAG? **Yes.** To check $A \perp\!\!\!\perp B \mid S$:
  1. Find all paths between A and B.
  2. Check whether each path is **blocked**, given that we condition on S.
  3. If every path is blocked, A and B are **d-separated** by S, and $A \perp\!\!\!\perp B \mid S$.

- A path is **blocked** given S if
  1. it contains a **non-collider** (fork or chain middle) that is in S, **or**
  2. it contains a **collider** that is not in S and has no descendant in S.

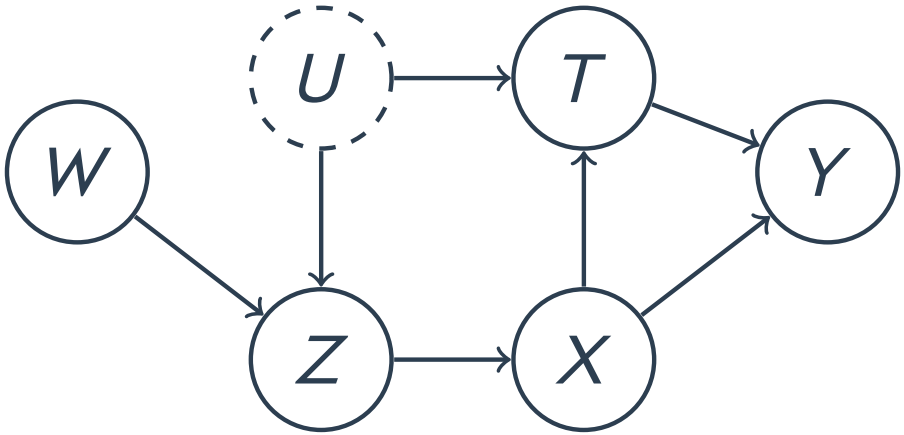
- If some path stays open, A and B are **d-connected**: expect them to move together.

Descendants of a Collider Count Too



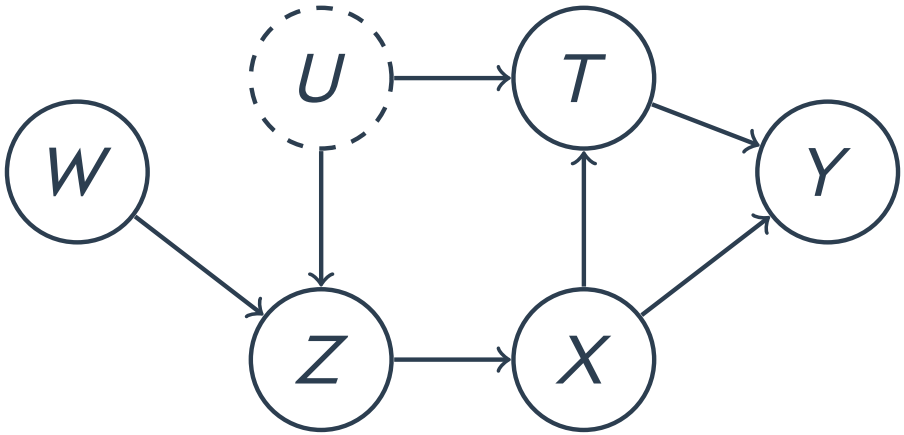
- C: loyalty member. N: receives the members' newsletter (only members get it).
- Analyzing only newsletter recipients = selecting on a descendant of the collider.
- The path $T \rightarrow C \leftarrow Y$ opens just as if we had selected on C.
- In practice: ask **how a row got into your dataset**. Survey responders, app reviewers, surviving firms, customers who log in.

d-Separation Example



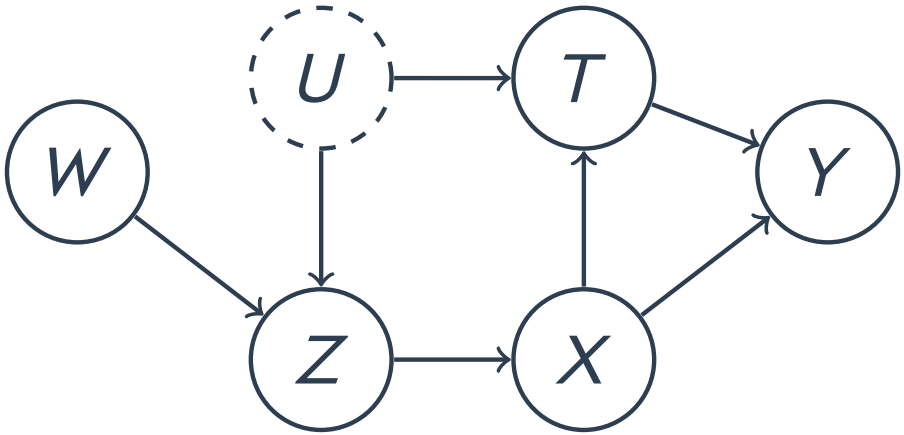
- Q1: Are W and Y independent of each other?

d-Separation Example



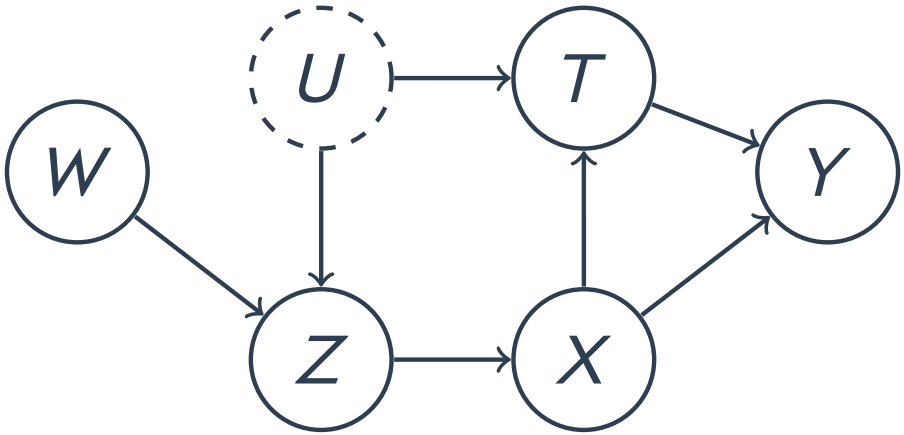
- Q1: Are W and Y independent of each other?
- No.
  - Blocked at the collider Z: $W \to Z \leftarrow U \to T \to Y$, $W \to Z \leftarrow U \to T \leftarrow X \to Y$
  - Open: $W \to Z \to X \to Y$, $W \to Z \to X \to T \to Y$

d-Separation Example



- Q1: Are W and Y independent of each other?
- No.
  - Blocked at the collider Z: $W \to Z \leftarrow U \to T \to Y$, $W \to Z \leftarrow U \to T \leftarrow X \to Y$
  - Open: $W \to Z \to X \to Y$, $W \to Z \to X \to T \to Y$
- Q2: Which variables should we condition on to make W and Y d-separated?

d-Separation Example



- Q1: Are W and Y independent of each other?
- No.
  - Blocked at the collider Z: $W \to Z \leftarrow U \to T \to Y$, $W \to Z \leftarrow U \to T \leftarrow X \to Y$
  - Open: $W \to Z \to X \to Y$, $W \to Z \to X \to T \to Y$
- Q2: Which variables should we condition on to make W and Y d-separated?
- $\{X, T\}$. X blocks both open paths but, as a descendant of the collider Z, opens $W \to Z \leftarrow U \to T \to Y$; T blocks that path.